

When Does Hierarchical Retrieval Beat Vector Search?

A Cross-Domain Empirical Study

Shivam Rawat

Abstract—Retrieval-Augmented Generation (RAG) pipelines typically rely on dense vector similarity to locate relevant document chunks. PageIndex, a recent alternative, organizes documents into hierarchical trees and uses LLM-guided traversal for retrieval, reporting 98.7% accuracy on the FinanceBench benchmark—but from a single domain under the authors’ own evaluation. We compare three retrieval strategies across four document QA domains (600 questions total) with a shared LLM backbone that isolates retrieval as the sole variable. Our central finding: hierarchical retrieval’s apparent large advantage on financial documents (judge score 3.94 vs. 1.69 for dense RAG, $d=1.16$) is primarily a retrieval coverage artifact. Dense RAG returns “NOT FOUND” for 53% of financial questions; hierarchical retrieval always selects content from the tree. When we restrict to questions both systems answered, the gap collapses from 2.25 to 0.31 points. Conversely, dense RAG dominates on short technical passages (3.93 vs. 1.75, $d=-1.09$), where hierarchical retrieval’s tree structure adds overhead without benefit. No system wins overall ($p=0.58$). We argue that the hierarchical-vs-vector choice is better understood as a coverage-precision tradeoff, modulated by document length and structural richness, than as an accuracy comparison.

Index Terms—Retrieval-augmented generation, document question answering, hierarchical retrieval, PageIndex, information retrieval, LLM evaluation

I. INTRODUCTION

Ask a RAG system to find a specific number in a 200-page financial report and it will often fail. The problem is well-studied but stubbornly persistent: when a question targets a particular line item—say, Walmart’s days payable outstanding in FY2018—the relevant chunk may sit in a table with headers like “Accounts Payable” and “Cost of Goods Sold,” sharing almost no semantic overlap with the question. A 1000-character chunk around that table row just does not embed close to “What is the DPO?” in vector space.

PageIndex [1] tries something different. It builds a tree over the document—leaves hold paragraph-level text, parents hold LLM-generated summaries of their children, and a root node summarizes everything. At query time, the LLM walks this tree from the top, picking branches that look relevant to the question. VectifyAI reports 98.7% accuracy on FinanceBench [2] with this approach, which is striking. But the evaluation covers one domain, run by the people who built the system.

S. Rawat is with Microsoft Azure, Redmond, WA 98052, USA. E-mail: shivarawat24@gmail.com. This work was conducted independently and does not represent the views of Microsoft Corporation.

That is a strong number, and if it holds across other document types, it would be a clear win for hierarchical retrieval. Three questions seem worth asking:

- RQ1.** Does hierarchical retrieval outperform vector-based retrieval beyond finance—on legal, scientific, and technical documents?
- RQ2.** Under what document characteristics (length, structural richness) does each approach win?
- RQ3.** What is the token cost per correct answer for each approach?

We ran the experiment. Three retrieval systems, four domains, 150 questions each, same LLM everywhere. The result that surprised us most was not about which system “wins” (no system does, overall: $p=0.58$). It was about *why* one system looks like it wins on finance. The apparent large effect ($d=1.16$) is mostly an artifact of retrieval coverage: hierarchical retrieval always pulls out some text from the tree, while dense RAG says “NOT FOUND” 53% of the time when no chunk is similar enough. Filter out those NOT FOUND cases and the gap drops from 2.25 points to 0.31. That changes the story considerably.

So the real question for practitioners is not “which method is more accurate?” but “which coverage-precision tradeoff fits my documents?” We think that framing is more honest about what the numbers show.

Our contributions:

- 1) To our knowledge, the first independent cross-domain evaluation of hierarchical tree retrieval vs. dense RAG, spanning four document types.
- 2) A coverage-conditioned analysis revealing that raw effect sizes from single-domain evaluations are primarily driven by retrieval coverage differences, not answer quality.
- 3) A cost-efficiency analysis normalizing token expenditure by accuracy, showing that the most efficient system depends on the domain.
- 4) An error taxonomy ($n=102$) categorizing failure modes by retrieval strategy.

II. BACKGROUND AND RELATED WORK

A. Retrieval-Augmented Generation

Lewis et al. [3] introduced RAG as a way to ground LLM responses in retrieved evidence. The standard pipeline is straightforward: chunk the document, encode chunks with a dense model [4], throw them into FAISS [5], and retrieve

the top- k at query time. Since 2020, dense retrieval has gotten much better. ColBERT [15] introduced late interaction; ColBERTv2 [16] made it practical. BGE-M3 [17] merged sparse and dense signals into one model. HyDE [18] took a different tack entirely, generating a hypothetical answer first and using *that* as the query embedding. We mention these because the coverage problems we find for basic dense RAG might look different with newer embeddings—something we revisit in Section V-C.

B. BM25 and Sparse Retrieval

Robertson and Zaragoza [6] formalized BM25, and it refuses to die. Term frequency, inverse document frequency, no neural anything—yet it regularly beats dense retrieval on specialized corpora [7]. The reason is simple: when a query contains a rare domain term like “days payable outstanding,” BM25 matches it directly. Embedding models often do not.

C. Hierarchical Tree Retrieval

PageIndex [1] takes a different approach to indexing. During construction, it chunks a document into leaf nodes, then has an LLM merge and summarize adjacent pairs to form parent nodes, repeating upward until it reaches a single root. At query time, the LLM reads the root summary, picks a branch, reads that summary, picks again, and eventually lands on a leaf.

RAPTOR [19] had a similar idea, independently. The main difference: RAPTOR clusters chunks rather than merging adjacent pairs, and at retrieval time it searches across all tree levels at once instead of walking top-down. Sarthi et al. saw gains over flat retrieval on NarrativeQA and QASPER. We followed PageIndex’s top-down approach but focused on a different question—not “does tree retrieval work?” but “why does it appear to work so much better on some domains?”

Self-RAG [20] is relevant here too, though from a different angle. It lets the LLM decide whether to retrieve at all. That connects to our finding that coverage—does the system even *try* to answer?—matters as much as the quality of the answer when it does.

One thing worth flagging about the architecture difference: vector retrieval does one similarity computation over all chunks. Tree retrieval makes multiple LLM calls, one per level. That is slower, but it can be more precise when the document has clear internal structure—sections, subsections, tables—that the tree mirrors.

D. Evaluation

We follow the LLM-as-judge approach [11]: a stronger model scores each answer on a 0–4 rubric. Several RAG evaluation frameworks exist—RAGAS [8], FRAMES [9], CRUD-RAG [10]—but our focus is on comparing retrieval strategies, not evaluation methods.

III. METHODOLOGY

A. Systems Under Comparison

All three systems share an identical LLM backbone (GPT-4o-mini, OpenAI, accessed via GitHub Copilot API) with

TABLE I
SYSTEM COMPARISON. ALL THREE SHARE THE SAME GENERATION LLM.

Component	BM25	Dense RAG	Hierarchical
Indexing	TF-IDF statistics	Sentence embeddings	LLM-generated tree
Retrieval	BM25 top-5	FAISS cosine top-5	LLM tree traversal
Embedding	—	all-MiniLM-L6-v2	—
Generation	Same LLM	Same LLM	Same LLM

temperature=0 and max_tokens=256. The only difference is how context reaches the LLM. Table I summarizes the design.

The hierarchical system replicates PageIndex’s algorithm as described in Section 3 of [1]: documents split into leaf nodes, adjacent pairs merged and summarized by the LLM to form parent nodes, parent summaries merged into a root. We implemented this ourselves rather than using the official codebase—a limitation we discuss in Section V-C. Tree construction required 370 LLM summarization calls across all four domains.

B. Datasets

We selected four domains to span a range of document types (Table II).

TABLE II
DATASET SUMMARY. STRUCTURAL RICHNESS MEASURES THE NORMALIZED DENSITY OF SECTION HEADERS, BULLETS, AND NUMBERED LISTS PER LINE (0=PLAIN PROSE, 1=HEAVILY STRUCTURED).

Domain	Source	n	Avg. Length	Structure
Finance	FinanceBench [2]	150	5,512 ch	0.00
Legal	CUAD [12]	150	4,958 ch	0.10
Science	QASPER [13]	150	7,918 ch	0.98
Technology	SQuAD-Tech*	150	812 ch	0.00

*SQuAD v2 [14] filtered by technology keywords (author-created subset).

Sampling was stratified by document length (three tertiles) with a fixed seed (42). The technology domain uses SQuAD v2 filtered by technology keywords rather than IBM’s TechQA, which was unavailable through standard channels. The resulting passages are substantially shorter than real technical documentation—a limitation we address in Section V-C.

C. Evaluation Protocol

Primary metric: LLM-as-Judge. A separate, stronger LLM independently scored each predicted answer against the gold answer on a 0–4 scale (4=fully correct; 3=mostly correct; 2=partially correct; 1=marginally relevant; 0=wrong or hallucinated). We chose LLM-as-judge over pure F1 because F1 penalizes correct paraphrases [11]. We report F1 and exact match as supplementary metrics.

Judge reliability. We double-scored a 10% holdout (60 answers) in a second independent pass. Overall weighted Cohen’s κ =0.98, Spearman ρ =0.97. We note that score distributions are bimodal (concentrated at 0 and 4), which inflates

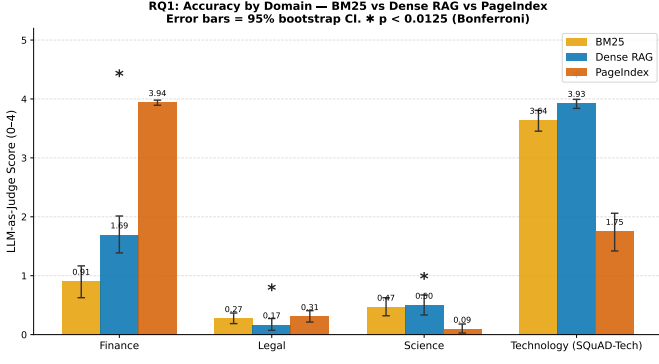


Fig. 1. Mean judge score by domain and system (error bars = 95% bootstrap CI). Stars mark statistically significant differences between hierarchical and dense RAG after Bonferroni correction.

agreement on extreme cases. We could not independently assess reliability on borderline scores (1–3) due to small borderline sample sizes.

Binary accuracy. Given the bimodality, we also report the fraction of answers scoring ≥ 3 as a more robust measure than the ordinal mean.

D. Statistical Analysis

We use paired Wilcoxon signed-rank tests with Bonferroni correction for four simultaneous domain tests (adjusted $\alpha=0.0125$). Effect sizes are Cohen’s d . Bootstrap 95% confidence intervals use 10,000 resampling iterations (seed=42).

Power. At the Bonferroni-corrected $\alpha=0.0125$, detecting $d=0.3$ with 80% power requires $n \approx 130$. Our $n=150$ per domain exceeds this threshold.

IV. RESULTS

A. RQ1: Accuracy Across Domains

Table III presents the main comparison. All four domain-level tests are significant after Bonferroni correction, but the direction of the effect reverses across domains. Hierarchical retrieval wins on finance ($d=1.16$, large) and shows a small edge on legal ($d=0.16$). Dense RAG wins on science ($d=-0.36$, small) and technology ($d=-1.09$, large). The overall comparison is not significant ($p=0.58$, $d=-0.02$).

Binary accuracy sharpens the picture. Hierarchical retrieval achieves 99.3% on finance but only 43.3% on technology; dense RAG achieves 98.0% on technology but only 42.0% on finance. Fig. 1 visualizes the domain-level accuracy with confidence intervals.

B. Decomposing the Effect: Coverage vs. Quality

The large finance effect ($d=1.16$) is the headline number, but Table IV shows it is driven largely by a confound: retrieval coverage.

Dense RAG returns NOT FOUND for 53% of financial questions. Every one of those scores zero, dragging the unconditional mean down to 1.69. When we look at just the 70 questions RAG actually answered, its mean jumps to 3.63. To control for selection effects, we also computed hierarchical

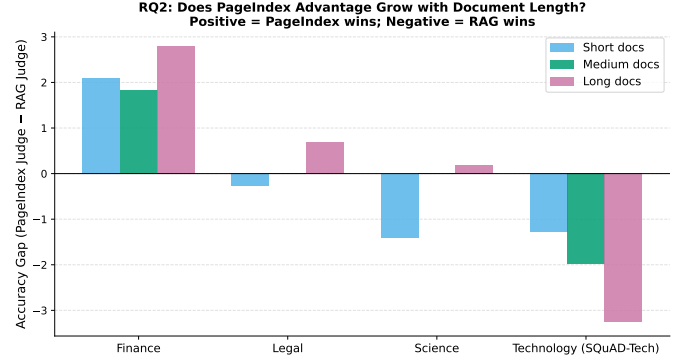


Fig. 2. Accuracy gap (hierarchical – RAG judge score) by document length tercile. Hierarchical retrieval’s advantage grows with document length on three of four domains.

retrieval’s score on those same 70 questions: 3.94. The gap on the common answered set is 0.31—a far cry from the raw 2.25.

The same pattern flips on technology. There, hierarchical retrieval returns NOT FOUND for 55% of questions, while dense RAG answers 99.3%. On the questions hierarchical retrieval does answer, it scores 3.87—essentially matching RAG’s 3.95.

What this boils down to: the two approaches differ in *when they can retrieve*, not *how well they answer*. Tree traversal handles structured, long documents where coarse summaries can route the LLM to the right section. Dense search handles short, keyword-rich passages where embedding similarity is enough.

C. RQ2: Document Characteristics

Table V shows Spearman correlations between document characteristics and the per-question accuracy gap (hierarchical minus RAG judge score).

In three of four domains, longer documents correlate with a larger hierarchical retrieval advantage (Fig. 2). Technology is the exception: documents are so short (mean 812 characters) that dense retrieval can capture the whole passage in a single chunk.

D. RQ3: Token Cost

Hierarchical retrieval uses $0.71\text{--}0.99\times$ the tokens of dense RAG per question (Fig. 3), because tree traversal selects a single node’s content rather than concatenating five chunks. But raw token cost is misleading without normalizing for accuracy. When we compute tokens per correct answer (Table VI), hierarchical retrieval is most cost-efficient on finance (506 vs. 1,683 for RAG) while dense RAG wins on technology (765 vs. 1,281).

E. Failure Modes

We sampled and classified 102 failure cases (answers scoring 0–1, up to 10 per domain-method cell) into six categories using an LLM classifier. Entity lookup failures account for

TABLE III
RQ1: ACCURACY BY DOMAIN (LLM-AS-JUDGE, 0–4 SCALE). BOLD = BEST PER DOMAIN. 95% BOOTSTRAP CIS. ALL FOUR DOMAIN TESTS SIGNIFICANT AT BONFERRONI $\alpha=0.0125$; OVERALL NOT SIGNIFICANT.

Domain	n	BM25	Dense RAG	Hierarchical	p (Hier. vs RAG)	Cohen's d
Finance	150	0.91 [0.63, 1.17]	1.69 [1.39, 2.01]	3.94 [3.89, 3.98]	<0.001	1.16 (large)
Legal	150	0.27 [0.19, 0.37]	0.17 [0.07, 0.27]	0.31 [0.21, 0.41]	0.008	0.16 (small)
Science	150	0.47 [0.32, 0.63]	0.50 [0.33, 0.67]	0.09 [0.03, 0.18]	<0.001	−0.36 (small)
Technology	150	3.64 [3.45, 3.81]	3.93 [3.84, 3.99]	1.75 [1.42, 2.06]	<0.001	−1.09 (large)
Overall	600	1.32 [1.18, 1.47]	1.57 [1.43, 1.72]	1.52 [1.38, 1.68]	0.577	−0.02

TABLE IV
COVERAGE-ADJUSTED SCORES (ANSWERED-ONLY). THE RAW GAP OF 2.25 ON FINANCE COLLAPSES TO 0.31 WHEN NOT FOUND ANSWERS ARE EXCLUDED.

Domain	RAG NF%	Hier. NF%	RAG adj.	Hier. adj.
Finance	53.3%	0.0%	3.63 (70)	3.94 (150)
Legal	24.7%	54.7%	0.22 (113)	0.68 (68)
Science	48.0%	74.0%	0.96 (78)	0.36 (39)
Technology	0.7%	54.7%	3.95 (149)	3.87 (68)

NF% = NOT FOUND rate. Adj. = mean score on answered questions only. (r

TABLE V
RQ2: DOCUMENT CHARACTERISTIC CORRELATIONS. POSITIVE r = HIERARCHICAL RETRIEVAL DOES BETTER ON DOCUMENTS WITH THAT CHARACTERISTIC.

Domain	$r(\text{length})$	p	$r(\text{structure})$	p
Finance	0.09	0.262	0.20	0.014
Legal	0.19	0.019	−0.58	<0.001
Science	0.66	<0.001	0.70	<0.001
Technology	−0.60	<0.001	0.16	0.045

70% of all errors, consistent across systems. BM25 is especially prone to these (31 of 40 BM25 failures) because entity names in the question may not match the document's terminology. Dense RAG shows more numerical reasoning failures (8 of 32 vs. 3 of 27 for hierarchical), possibly because tree traversal preserves the context surrounding tables where numbers appear.

V. DISCUSSION

A. The Coverage-Precision Tradeoff

The numbers in Section IV point to a structural explanation. Hierarchical traversal forces the LLM to commit to a branch at every tree level. It always ends up at a leaf node with text—that is how trees work. Whether that text is relevant is another question. Dense RAG, by contrast, can fail gracefully: if no chunk is similar enough, the generation LLM says so. One approach guarantees coverage at the cost of precision; the other guarantees precision at the cost of coverage.

On financial reports, coverage wins. The documents are long, they have clear sections (Income Statement, Balance Sheet, Notes), and the tree structure mirrors this organization. On short tech passages, coverage is free—dense retrieval can already find the answer 99% of the time—so the tree just

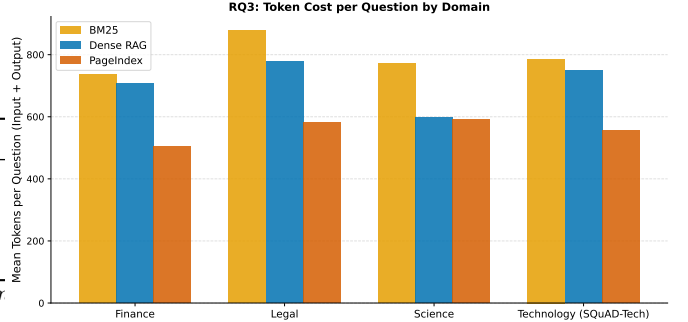


Fig. 3. Mean tokens per question (input + output) by domain and system. Hierarchical retrieval uses fewer tokens because it selects one tree node rather than concatenating five chunks.

TABLE VI
RQ3: TOKENS PER CORRECT ANSWER (TOKENS/QUESTION ÷ BINARY ACCURACY). LOWER = MORE EFFICIENT.

Domain	BM25	RAG	Hier.	Best
Finance	3,242	1,683	506	Hier.
Legal	—	38,850	29,000	Hier.
Science	16,552	7,475	29,550	RAG
Technology	861	765	1,281	RAG

— = binary accuracy below 1%.

adds latency. Fig. 4 plots each domain in the accuracy-gain vs. cost-ratio space, making the tradeoff visible at a glance.

B. What This Means in Practice

The recommendation depends on what you are building. Long, structured documents with section headings (financial reports, regulatory filings, big technical manuals)? Hierarchical retrieval's coverage advantage probably justifies the extra indexing cost. Short documents, or documents you expect to chunk cleanly? Dense retrieval is simpler and works fine.

A caveat, though. We used all-MiniLM-L6-v2 for the dense embeddings—a 22-million-parameter model released in 2021. That is not what anyone would use today. BGE-large, E5-mistral, GTE-Qwen2: any of these would produce better embeddings, and hybrid systems that combine BM25 with dense signals [17] would further close the gap. Our experiment tests the *architectural* tradeoff (tree vs. flat), not any particular implementation. A modern dense pipeline might not have a 53% NOT FOUND rate on financial questions. We do not know; we did not test it.

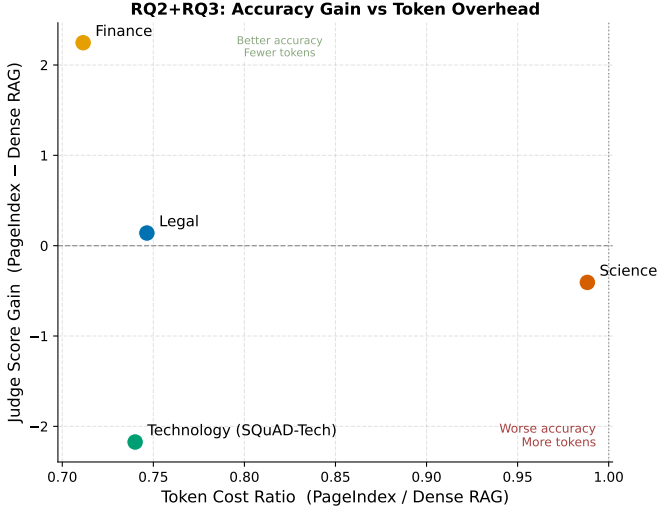


Fig. 4. Accuracy gain (hierarchical — RAG) vs. token cost ratio (hierarchical / RAG) by domain. Finance sits in the upper-left quadrant (better accuracy, fewer tokens); Technology sits in the lower-right.

It is also worth being upfront about the legal and science domains. All three systems scored below 0.50 on both. At that level of performance, we are comparing noise, not systems.

C. Limitations

The biggest threat to our conclusions is that we did not use PageIndex’s actual code. We reimplemented their algorithm from the paper description, and our tree construction may well differ from theirs in ways we cannot verify. Testing with the official codebase, and with RAPTOR [19], is the obvious next step.

Our dense retrieval pipeline is also dated. All-MiniLM-L6-v2 is small, old, and not representative of what a modern RAG stack would use. The 53% NOT FOUND rate on finance might drop substantially with better embeddings or larger chunks. (We used 1,000-character chunks throughout, and chunk boundaries that split a table in half will obviously hurt retrieval.)

The technology dataset is a weak point. SQuAD-Tech passages average 812 characters—short enough that a single chunk often contains the entire passage. Dense RAG’s 98% accuracy there probably reflects how easy the task is, not some intrinsic advantage of embedding-based retrieval on technical content.

Two more things worth stating plainly. First, we used one LLM for generation and one for judging. A stronger generator might compensate for bad retrieval; a different judge might score differently. Second, we report token cost but not latency, and for production use the sequential LLM calls in tree traversal are a real concern that our cost-per-question numbers do not capture.

VI. CONCLUSION

We tested three retrieval strategies on 600 questions across four domains. None won overall. Hierarchical tree retrieval dominated on structured financial documents; dense RAG

dominated on short technical passages. The thing that surprised us, and that we think matters most for practitioners, is the coverage decomposition: strip out the NOT FOUND cases and the accuracy gap between systems nearly vanishes. The systems are not really competing on answer quality—they are competing on whether they can find the answer at all. Which one you should pick depends on your documents.

ACKNOWLEDGMENTS

Portions of the experimental pipeline were orchestrated using GitHub Copilot CLI. All results, analysis, and writing were verified by the author. We thank the anonymous reviewers for their feedback.

REFERENCES

- [1] VectifyAI, “PageIndex: An efficient hierarchical document index for LLM-based retrieval,” *arXiv:2407.01870*, 2024.
- [2] S. Islam et al., “FinanceBench: A new benchmark for financial question answering,” *arXiv:2311.11944*, 2023.
- [3] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020.
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP*, 2019.
- [5] J. Johnson et al., “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [6] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
- [7] J. Lin, “Pretrained transformers for text ranking: BERT and beyond,” *Synthesis Lectures on HLT*, vol. 14, no. 4, 2021.
- [8] S. Es et al., “RAGAS: Automated evaluation of retrieval augmented generation,” *arXiv:2309.15217*, 2023.
- [9] K. Krishna et al., “FRAMES: Benchmarking and evaluating open-domain RAG,” *arXiv*, 2024.
- [10] Y. Lyu et al., “CRUD-RAG: A comprehensive benchmark for retrieval-augmented generation,” *arXiv*, 2024.
- [11] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Proc. NeurIPS*, 2023.
- [12] D. Hendrycks et al., “CUAD: An expert-annotated NLP dataset for legal contract review,” in *Proc. NeurIPS*, 2021.
- [13] P. Dasigi et al., “A dataset of information-seeking questions and answers anchored in research papers,” in *Proc. NAACL*, 2021.
- [14] R. Rajpurkar et al., “Know what you don’t know: Unanswerable questions for SQuAD,” in *Proc. ACL*, 2018.
- [15] O. Khattab and M. Zaharia, “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT,” in *Proc. SIGIR*, 2020.
- [16] K. Santhanam et al., “ColBERTv2: Effective and efficient retrieval via lightweight late interaction,” in *Proc. NAACL*, 2022.
- [17] J. Chen et al., “BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings,” *arXiv:2402.03216*, 2024.
- [18] L. Gao et al., “Precise zero-shot dense retrieval without relevance labels,” in *Proc. ACL*, 2023.
- [19] P. Sarthi et al., “RAPTOR: Recursive abstractive processing for tree-organized retrieval,” in *Proc. ICLR*, 2024.
- [20] A. Asai et al., “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *Proc. ICLR*, 2024.



Shivam Rawat is a Software Engineer at Microsoft Azure, Redmond, WA, working on cloud infrastructure and AI systems. His research interests include information retrieval, retrieval-augmented generation, and empirical evaluation of LLM-based systems.